Fondements informatiques I

Cours 8: Modules et fichiers

Sorina Ionica sorina.ionica@uvsq.fr

Bilal FAYE bilal.faye@uvsq.fr

Pourquoi utiliser des modules ?

Imaginons que l'on souhaite programmer une petite application de calculatrice.
Le programme pourrait contenir :

- des fonctions pour effectuer des calculs ;
- des fonctions pour gérer l'affichage ;
- un programme principal qui utilise ces fonctions.

Problème

Regrouper toutes les fonctions dans un seul fichier peut rendre le programme difficile à lire et à maintenir.

Solution : les modules

On peut répartir les fonctions dans plusieurs fichiers Python, en regroupant celles qui ont un rôle commun.

Créer son propre module

Un **module** est un fichier Python portant l'extension `.py`.
On peut y définir des fonctions qui pourront être réutilisées dans d'autres programmes.

Fichier `calculs.py`

```
def carre(x):
    return x * x

def moyenne(a, b):
    return (a + b) / 2
```

- Le fichier `calculs.py` constitue un module.
- Il contient deux fonctions réutilisables.

Importer son propre module

Pour utiliser les fonctions du module `calculs`, on utilise l'instruction `import`.

Fichier `main.py`

```
import calculs

print(calculs.carre(5))
print(calculs.moyenne(10, 20))
```

- `import calculs` importe le module.
- On utilise ensuite le nom du module suivi d'un point pour accéder à ses fonctions.

Organisation des fichiers

Les deux fichiers `calculs.py` et `main.py` doivent être placés dans le même dossier pour cet exemple.

Importer directement une fonction

Il est également possible d'importer uniquement les fonctions dont on a besoin.

Fichier `main.py`

```
from calculs import carre

print(carre(5))
```

- `from calculs import carre` importe uniquement la fonction `carre`.
- On utilise directement son nom, sans préfixer par `calculs.`.

À retenir

`import calculs` permet d'utiliser le module entier.
`from calculs import carre` permet d'importer une fonction particulière.

Quelques modules de Python

Python fournit de nombreux modules contenant des fonctions prêtes à être utilisées.

Le module math

Il contient des fonctions et constantes mathématiques.

```
import math

print(math.sqrt(25))   # Racine carree
print(math.pi)         # Constante pi
```

Le module random

Il permet de générer des valeurs aléatoires.

```
import random

nombre = random.randint(1, 10)
print(nombre)
```

Importer une fonction d'un module

On peut également importer directement une fonction d'un module fourni par Python.

Exemple avec le module `math`

```
from math import sqrt, pi

print(sqrt(49))
print(pi)
```

Une pratique à éviter

L'instruction `from math import *` importe de nombreux noms sans préciser leur origine. Il est préférable d'utiliser `import math` ou d'importer explicitement les fonctions nécessaires.

Pourquoi utiliser des fichiers ?

Jusqu'à présent, nous avons stocké les informations dans des variables.
Mais que deviennent ces informations lorsque le programme s'arrête ?

- Les variables disparaissent à la fin de l'exécution.
- Les fichiers permettent de conserver des informations sur le disque.
- Ces informations pourront être relues lors d'une prochaine exécution du programme.

Exemples d'utilisation

Enregistrer les notes d'un élève, conserver une liste de noms ou sauvegarder un résultat de calcul.

Écrire dans un fichier

La fonction `open()` permet d'ouvrir un fichier.
Pour écrire dans un fichier, on utilise le mode `"w"`.

Écrire dans `notes.txt`

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Alice : 15\n")
    f.write("Bob : 12\n")
```

- `"w"` : ouvre le fichier en écriture.
- `write()` : écrit du texte dans le fichier.
- `\n` : permet de passer à la ligne.

Attention

Le mode `"w"` crée le fichier s'il n'existe pas. S'il existe déjà, son ancien contenu est effacé.

Lire un fichier

Pour lire le contenu d'un fichier, on utilise le mode `"r"`.
La méthode `read()` permet de récupérer tout son contenu.

Lire `notes.txt`

```
with open("notes.txt", "r", encoding="utf-8") as f:
    contenu = f.read()

print(contenu)
```

- `"r"` : ouvre le fichier en lecture.
- `read()` : récupère tout le texte du fichier.
- Le contenu peut ensuite être stocké dans une variable.

Ajouter du contenu dans un fichier

Pour ajouter du texte sans effacer le contenu existant, on utilise le mode `"a"`.

Ajouter une nouvelle note

```
with open("notes.txt", "a", encoding="utf-8") as f:
    f.write("Claire : 18\n")
```

Les trois modes à connaître

Mode	Utilisation
`"r"`	Lire un fichier
`"w"`	Écrire (remplace le contenu)
`"a"`	Ajouter à la fin du fichier